

PLP - Primer Parcial - 1^{er} cuatrimestre de 2025 - Parte 2

#Orden	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final
		Pérez Natalia	E	E	E	P

(Promocionado)

Las notas para cada ejercicio son: -, I, R, B, MB, E. Este examen se aprueba obteniendo al menos dos ejercicios B y uno R, y se promociona con al menos dos ejercicios MB y uno B. Es posible obtener una aprobación condicional con un ejercicio MB, uno B y uno I. Esto requiere entregar algo que contribuya a la solución del ejercicio. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación funcional

Aclaración: en este ejercicio no está permitido utilizar recursión explícita, a menos que se indique lo contrario.

Se define el tipo de datos ABNV a, que representa árboles binarios no vacíos con elementos de tipo a:

data ABNV a = Hoja a | Uni a (ABNV a) | Bi (ABNV a) a (ABNV a)

Definimos el siguiente árbol para los ejemplos:

abnv = Bi (Uni 2 (Hoja 1)) 3 (Bi (Hoja 4) 5 (Uni 2 (Hoja 7)))

- Definir las funciones foldABNV y recABNV, que implementan respectivamente los esquemas de recursión estructural y primitiva para el tipo ABNV a. Solo en este inciso se permite usar recursión explícita.
- Definir la función elemABNV :: Eq a => a -> ABNV a -> Bool, que indica si un elemento pertenece a un árbol.
Por ejemplo: elemABNV 7 abnv ~ True.
- Definir la función reemplazarUno :: Eq a => a -> a -> ABNV a -> ABNV a que, dados dos elementos x e y y un árbol, devuelve un árbol como el original, pero reemplazando la primera aparición de x por y (la primera desde la raíz yendo de izquierda a derecha, en el orden de preorder).
Por ejemplo:
reemplazarUno 2 5 abnv ~ Bi (Uni 5 (Hoja 1)) 3 (Bi (Hoja 4) 5 (Uni 2 (Hoja 7))).
reemplazarUno 2 5 (Hoja 1) ~ Hoja 1.
- Definir la función nivel :: ABNV a -> Int -> [a], que liste todos los elementos del nivel indicado, de izquierda a derecha, siendo 0 el nivel de la raíz. Si el nivel no existe (ya sea por ser negativo o por superar la altura del árbol), devolver [].

Por ejemplo:

nivel abnv 1 ~ [2,5].

nivel abnv 2 ~ [1,4,2].

nivel abnv 4 ~ [].

Pista: aprovechar la curificación y utilizar evaluación parcial.

Ejercicio 2 - Demostración de propiedades

- Considerar las siguientes definiciones:

```
data AB a = Nil | Bin (AB a) a (AB a)
```

```
elemAB :: Eq a => a -> AB a -> Bool
```

```
{P0} elemAB _ Nil = False
```

```
{P1} elemAB x (Bin i r d) = (x == r) || (elemAB x i) || (elemAB x d)
```

```
mapAB :: (a -> b) -> AB a -> AB b
```

```
{M0} mapAB _ Nil = Nil
```

```
{M1} mapAB f (Bin i r d) = Bin (mapAB f i) (f r) (mapAB f d)
```


Sean dos tipos a y b tales que valen $\text{Eq } a$, $\text{Eq } b$, y por ende también la siguiente propiedad:

$$\{\text{CONGRUENCIA} ==\} \forall x::a. \forall y::a. \forall f::a \rightarrow b. x == y \Rightarrow f x == f y$$

Demostrar la siguiente propiedad:

$$\forall t::AB \ a. \forall x::a. \forall f::a \rightarrow b. (\text{elemAB } x \ t \Rightarrow \text{elemAB } (f \ x) \ (\text{mapAB } f \ t))$$

Se permite definir macros (i.e., poner nombres a expresiones largas para no tener que repetirlas).

No es obligatorio escribir los $\forall$ correspondientes en cada paso, pero es importante recordar que están presentes. Recordar también que los $=$ de las definiciones pueden leerse en ambos sentidos.

Se consideran demostradas todas las propiedades conocidas sobre enteros y booleanos.

Sugerencia: tener en cuenta que $\forall x::\text{Bool}. \forall y::\text{Bool}. \forall z::\text{Bool}. (x \Rightarrow y) \Rightarrow (x \Rightarrow z \vee y)$.

b) Demostrar el siguiente teorema usando Deducción Natural, sin utilizar principios clásicos:

$$\rho \Rightarrow (\sigma \vee (\rho \Rightarrow \tau)) \Rightarrow (\sigma \vee \tau)$$

Ejercicio 3 - Cálculo Lambda Tipado

Se desea extender el cálculo lambda simplemente tipado para modelar **listas ordenadas**. Una lista ordenada se construye de manera similar a una lista común, pero sus observadores se comportan de manera diferente: la cabeza de la lista es siempre el mínimo elemento, y su cola es el resto de los elementos.

Estas listas tienen como precondition que el tipo de sus elementos cuente con una operación de comparación $<$ (no es necesario escribir nada al respecto, suponemos que vale).

Se extienden los tipos y expresiones de la siguiente manera:

$$\tau ::= \dots \mid [\tau]^<$$

$$M ::= \dots \mid []_\tau^< \mid M ::^< M \mid \text{head}^<(M) \mid \text{tail}^<(M)$$

donde:

- $[\tau]^<$ es el tipo de las listas ordenadas con elementos de tipo τ .
- $[]_\tau^<$ es una lista ordenada vacía que admite valores de tipo τ .
- $M_1 ::^< M_2$ es la lista resultante de agregar el elemento M_1 a la lista M_2 ¹.
- $\text{head}^<(M)$ denota el mínimo elemento de la lista M .
- $\text{tail}^<(M)$ es la lista ordenada con todos los elementos de M excepto el mínimo (si M tiene más de una aparición de su mínimo elemento, se elimina una de ellas).

a) Introducir las reglas de tipado para la extensión propuesta.

b) Definir el conjunto de valores y las nuevas reglas de reducción en un paso. Tener en cuenta que una lista vacía no tiene cabeza ni cola.

Pista: puede ser necesario mirar más de un nivel de un término para saber a qué reduce.

c) Mostrar paso por paso cómo reduce la expresión:

$$\text{head}^<((\lambda x: \text{Nat}. 2)\text{zero}) ::^< 1 ::^< []_{\text{Nat}}^<$$

Se pueden considerar dadas las siguientes reglas:

$$\frac{M_1 \rightarrow M'}{M_1 < M_2 \rightarrow M' < M_2} (E-<_1) \quad \frac{M_2 \rightarrow M'}{V < M_2 \rightarrow V < M'} (E-<_2)$$

$$\frac{}{V < \text{zero} \rightarrow \text{False}} (E-<_F) \quad \frac{}{\text{zero} < \text{Succ}(V) \rightarrow \text{True}} (E-<_T)$$

$$\frac{}{\text{Succ}(V_1) < \text{Succ}(V_2) \rightarrow V_1 < V_2} (E-<_S)$$

¹Notar que M_1 no necesariamente es la cabeza de la lista, y que $::^<$ es un constructor, no es responsable de ordenar los elementos en la lista. El orden aparece al observarla. Por ejemplo, $1 ::^< 2 ::^< []_{\text{Nat}}^<$ y $2 ::^< 1 ::^< []_{\text{Nat}}^<$ son dos valores de tipo $[\text{Nat}]^<$ que tienen la misma cabeza y la misma cola.

Ej 1:

a

~~foldABNV :: (a → b) → (a → b → b) → (b → a → b → b) → ABNV a → b~~
~~foldABNV fHoja fUni fBi~~

foldABNV :: (a → b) → (a → b → b) → (b → a → b → b) → ABNV a → b

foldABNV fHoja fUni fBi ab = case ab of

Hoja v → fHoja v

Uni v ar → fUni v (rec ar)

Bi i v d → fBi (rec i) v (rec d)

where

rec = foldABNV fHoja fUni fBi

~~recABNV :: (a → b) → (a → b → b) → (b → a → b → b) → ABNV a → b~~

recABNV :: (a → b) → (a → ABNV a → b → b) → ^{*}(ABNV a → b → a →

^{*}(ABNV a → b → a → ABNV a → b → b) → ABNV a → b

recABNV fHoja fUni fBi ab = case ab of

Hoja v → fHoja v

Uni v ar → fUni v ar (rec ar)

Bi i v d → fBi i (rec i) v d (rec d)

where

rec = recABNV fHoja fUni fBi

b elemABNV :: Eq a ⇒ a → ABNV a → Bool

elemABNV e = foldABNV (λv → if (v == e) then True

(λv rec → (v == e) || rec) (λrec i v recd → (v == e) || rec i || recd)

c reemplazarUno :: Eq a ⇒ a → a → ABNV a → ABNV a

reemplazarUno x y = recABNV (λv → if (x == v) then

(λv → if (x == v) then (Hoja y) else (Hoja v))

(λv ar rec → if (x == v) then (Uni y ar) else (Uni v rec))

(λi rec i v d recd → if (x == v) then (Bi i y d) else ^{*}(Bi i v recd))

^{*}(if (elemABNV x i) then (Bi i y d) else (Bi i v recd)))

Imaginar que todo
esté en una línea.

d nivel :: ABNV a → Int → [a]

nivel = foldABNV (\v → (\n → if (n == 0) then [v] else []))

~~nivel~~

(\v rec → (\n → if (n == 0) then [v] else ~~rec~~ rec (n-1))

(\reci v recd → (\n → if (n == 0) then [v] else ~~(reci (n-1) ++ recd (n-1))~~
(reci (n-1) ++ (recd (n-1))))

Ej 3:

a Reglas de tipado:

$$\frac{}{\Gamma \vdash []_T^< : [\tau]^<} \text{ T-VACIA} \quad \frac{\Gamma \vdash M_1 : \tau \quad \Gamma \vdash M_2 : [\tau]^<}{\Gamma \vdash M_1 ::^< M_2 : [\tau]^<} \text{ T-LISTA}$$

$$\frac{\Gamma \vdash M : [\tau]^<}{\Gamma \vdash \text{head}^<(M) : \tau} \text{ T-Head} \quad \frac{\Gamma \vdash M : [\tau]^<}{\Gamma \vdash \text{tail}^<(M) : [\tau]^<} \text{ T-tail}$$

b Conjunto de valores:

$$V ::= \dots \mid []_T^< \mid V ::^< V$$

Reglas de reducción:

$$\frac{M_1 \rightarrow M'_1}{M_1 ::^< M_2 \rightarrow M'_1 ::^< M_2} \text{ E-listad} \quad \frac{M \rightarrow M'}{V ::^< M \rightarrow V ::^< M'} \text{ E-lista2}$$

$$\frac{M \rightarrow M'}{\text{head}^<(M) \rightarrow \text{head}^<(M')} \text{ E-head} \quad \frac{M \rightarrow M'}{\text{tail}^<(M) \rightarrow \text{tail}^<(M')} \text{ E-tail}$$

~~head^<(V_1 ::^< V_2)~~

~~head^<(V_1)~~

~~head^<(V_1 ::^< V_2) \rightarrow \text{if } (V_1 < \text{head}(V_2)) \text{ then } V_1 \text{ else head}(V_2)~~

~~tail^<(V_1 ::^< V_2) \rightarrow \text{if } (V_1 < \text{head}(V_2)) \text{ then } V_2 \text{ else } V_1 ::^< \text{tail}(V_2)~~

head^<(V_1 ::^< []_T^<) \rightarrow V_1

head^<(V_1 ::^< V_2 ::^< V_3) \rightarrow \text{if } (V_1 < \text{head}(V_2 ::^< V_3)) \text{ then } V_1 \text{ else head}(V_2 ::^< V_3)

tail^<(V_1 ::^< []_T^<) \rightarrow []_T^<

tail^<(V_1 ::^< V_2 ::^< V_3) \rightarrow \text{if } (V_1 < \text{head}(V_2 ::^< V_3)) \text{ then } (V_2 ::^< V_3) \text{ else } V_1 ::^< \text{tail}(V_2 ::^< V_3)

(E-head)
vacio
(E-head)
lista

(E-tail)
vacio

(E-tail)
lista

C $\text{head}^<((\lambda x:\text{Nat}. 2) \text{zero}) ::^< 1 ::^< []_{\text{Nat}}^<)$

$\rightarrow \text{head}^<(\text{2})$

• E-head

β

$\text{head}^<(\text{2} ::^< 1 ::^< []_{\text{Nat}}^<) = \text{head}^<(\text{succ}(\text{succ}(\text{zero})) ::^< \text{succ}(\text{zero}) ::^< []_{\text{Nat}}^<)$

• E-head

• E-lista1

β

$\rightarrow \text{if } \text{succ}(\text{succ}(\text{zero})) < \text{head}^<(\text{succ}(\text{zero}) ::^< []_{\text{Nat}}^<) \text{ then } \text{succ}(\text{succ}(\text{zero}))$

$\text{else } \text{head}^<(\text{succ}(\text{zero}) ::^< []_{\text{Nat}}^<) = A$

• E-head
lista

$\rightarrow \text{if } \text{succ}(\text{succ}(\text{zero})) < \text{succ}(\text{zero}) \text{ A}$

• E-IF

• E-head

vacio + E-<2

$\rightarrow \text{if } \text{succ}(\text{zero}) < \text{zero} \text{ A}$

• E-IF

• E-<5

$\rightarrow \text{if false then succ}(\text{succ}(\text{zero})) \text{ else } \text{head}^<(\text{succ}(\text{zero}) ::^< []_{\text{Nat}}^<)$

• E-IF

• E-<f

$\rightarrow \text{head}^<(\text{succ}(\text{zero}) ::^< []_{\text{Nat}}^<) \rightarrow \text{succ}(\text{zero})$

• E-iffalse

E-head
vacio

Ej 2:

b sin principios clásicos.

$$\begin{array}{l}
 \text{ax} \quad \Gamma \vdash 6 \vee (P \Rightarrow T) \quad \Gamma, 6 \vdash 6 \quad \text{vii} \quad \Gamma, P \Rightarrow T \vdash T \quad \text{vii} \quad \Gamma, P \Rightarrow T \vdash 6 \vee T \quad \text{vi}_2 \\
 \Gamma \vdash 6 \vee (P \Rightarrow T) \quad \Gamma, 6 \vdash 6 \vee T \quad \Gamma, P \Rightarrow T \vdash 6 \vee T \quad \text{ve} \\
 \Gamma = \{P, (6 \vee (P \Rightarrow T))\} \vdash (6 \vee T) \quad \Rightarrow i \\
 P \vdash (6 \vee (P \Rightarrow T)) \Rightarrow (6 \vee T) \quad \Rightarrow i \\
 \vdash P \Rightarrow ((6 \vee (P \Rightarrow T)) \Rightarrow (6 \vee T))
 \end{array}$$

a Lo demuestro por inducción estructural en árboles ABA
 $\forall q \forall t :: AB \ a, P(t) \text{ vale.}$

$$\forall P(t) = \forall x :: a \quad \forall f :: a \rightarrow b \quad (\text{elem } AB \ x \ t \Rightarrow \text{elem } AB \ (fx)) \downarrow$$

$$\quad \quad \quad (\text{map } AB \ f \ t)$$

Por inducción estructural en AB a $Q \vee P$:

• $P(N=1)$ vale $\textcircled{1}$

- $P(\text{Nil})$ vale (1)
- $P(\text{Bin}) \forall i :: AB\ a, \forall d :: AB\ a, \forall r :: a \quad (P(i) \wedge P(d)) \Rightarrow P(\text{Bin } i\ r)$ (2)

①: $\text{Domain} = \text{set } x :: a, f :: a \rightarrow b \text{ Aug.}$

$$\text{elem } AB \times Nil \Rightarrow \text{elem } AB (f \ x) \ (\text{map } AB \ f \ Nil).$$
$$\text{elemAB} \times \text{Nil} = \text{False}$$

Entonces tengo $\text{false} \Rightarrow A$ y por prop. de Boole se que esto es verdadero siempre. $P(\text{Nil})$ vale $\checkmark$.

$$\forall i: A B a, \forall d: A B a;$$

② $P(i) = \forall x :: a \quad \forall f :: a \rightarrow b. (\text{elem } AB \ x \Rightarrow \text{elem } AB \ (f\ x)) \ (\text{map } AB \ f \ i)$

$$P(d) = \forall x :: a \quad \forall f :: a \rightarrow b. \text{elem } AB \ x \ d \Rightarrow \text{elem } AB \ (fx) \ (\text{map } AB \ f \ d)$$

son mi H.I (son verdaderos)

sea $x :: a$, $f :: a \rightarrow b$ ~~then~~, ~~then~~ $r :: a$ ~~then~~, $i, d :: AB\ a$

Ques: $\text{elemAB } x \text{ (Bin i r d)} \Rightarrow \text{elemAB } (f x) \text{ (mapAB } f \text{ (Bin i r d))}$

elemAB x (Bin i r d) lo asumo verdadero (ya que si es falso no tengo que probar nada) y veo que vale el consecuente.

$$\star \text{elemAB } x \text{ (Bin } i \text{ r d)} = (x == r) \parallel (\text{elemAB } x \text{ i}) \parallel (\text{elemAB } x \text{ d})$$

{PI}

consecuente:

$$\text{elemAB } (f x) \text{ (mapAB } f \text{ t)} = \text{elemAB } (f x) \text{ (Bin (mapAB } f \text{ i) (fr) (mapAB } f \text{ d))}$$

{PI}

$$\begin{aligned} & \text{elemAB } (f x) \text{ (Bin (mapAB } f \text{ i) (fr) (mapAB } f \text{ d))} \\ &= ((f x) == (f r)) \parallel (\text{elemAB } (f x) \text{ (mapAB } f \text{ i)}) \parallel \\ & \quad (\text{elemAB } (f x) \text{ (mapAB } f \text{ d)}) = \text{true} \end{aligned}$$

{PI}

Pro yo se que $\star$ es verdadero. Por lema de generación de Bool:

$$A) (x == r) = \text{True}$$

Por {congruencia ==} si $x == r \Rightarrow f x == f r$

Entonces:

$$((f x) == (f r)) \parallel B = \text{True} \parallel B \text{ y por prop de Bool}$$

$\text{True} \parallel B = \text{True} \Rightarrow$ El consecuente vale en este caso ✓

B) $(x == r) = \text{False}$. Entonces por lema de generación de Bool:

$$B.A) (\text{elemAB } x \text{ i}) = \text{True}$$

Por H.I tengo $Pci) = \text{elemAB } x \text{ i} \Rightarrow \text{elemAB } (f x) \text{ (map } f \text{ i)}$

entonces $\text{elemAB } (f x) \text{ (map } f \text{ i)} = \text{True}$

Tengo C $\parallel \text{elemAB } (f x) \text{ (mapAB } f \text{ d)}$

y por prop. de Bool $C \parallel \text{True} \parallel (\text{elemAB } (f x) \text{ (mapAB } f \text{ d)}) = \text{True}$
 $\Rightarrow$ el consecuente vale en este caso ✓

B.B) $(\text{elemAB } x \text{ i}) = \text{False}$, y como $\star$ es verdadero, entonces por prop de Bool, $\text{False} \parallel \text{False} \parallel A = \text{True} \Rightarrow A = \text{True}$.

entonces $\text{elemAB } x \text{ d}$ es verdadero y por H.I tengo

$Pcd) = \text{elemAB } x \text{ d} \Rightarrow \text{elemAB } (f x) \text{ (map } f \text{ d)}$ entonces

$\text{elemAB } (f x) \text{ (map } f \text{ d)}$ es verdadero. ✓

Pérez, Natalia

Tengo ~~map~~ $C \parallel (\text{elemAB } (f \ x) \ (\text{mapAB } f \ i)) \parallel \text{True}$

y por prop. de Bool $C \parallel (\text{elemAB } (f \ x) \ (\text{mapAB } f \ i)) \parallel \text{True} = \text{True}$
entonces de nuevo vale el consecuente ✓

Probé P que para cada caso, si vale el antecedente
entonces vale el consecuente $\Rightarrow$ probé la implicación.
 $P(\text{Bin } i \ r \ d)$ es verdadero.

~~Prueba por inducción~~

① y ② valen $\Rightarrow$ Por principio de inducción
estructural en $\text{AB } a$, queda demostrado que
 $\forall t :: \text{AB } a. P(t)$ es verdadero.

Esta parte del examen se aprueba respondiendo correctamente al menos el 75% de las preguntas. Es necesario aprobar esta parte para acceder a la corrección de la segunda parte. Responder las preguntas en los espacios indicados.

Pregunta 1: La siguiente definición ¿utiliza recursión explícita? Justificar.

```
esHoja :: AB a -> Bool
esHoja Nil = False
esHoja (Bin i r d) = esNil i && esNil d
  where esNil Nil = True
        esNil _ = False
```

Respuesta: NO, porque no se define ninguna función que se llame así misma. (no hay llamado recursivo).

Pregunta 2: Indicar el tipo del esquema de recursión estructural foldABNV para el siguiente tipo:

```
data ABNV a = Hoja a | Uni a (ABNV a) | Bi (ABNV a) a (ABNV a)
```

Respuesta: ~~foldABNV :: (a -> b) -> (a -> b -> b) -> (b -> a -> b -> b) -> ABNV a -> b~~
foldABNV :: (a -> b) -> (a -> b -> b) -> (b -> a -> b -> b) -> ABNV a -> b

Pregunta 3: ¿Qué tipo de recursión utiliza la siguiente definición? (Indicar la más específica).

```
listasQueSuman :: Int -> [[Int]]
listasQueSuman 0 = [[]]
listasQueSuman n | n > 0 = [x : xs | x <- [1..n], xs <- listasQueSuman (n-x)]
```

- | | |
|----------------|------------------------|
| a. Estructural | c. Global |
| b. Primitiva | d. Iterativa/a la cola |

Respuesta: C

Pregunta 4: Indicar cuál de las siguientes divisiones en casos puede justificarse por un Lema de Generación.

- | | | |
|---|---|--|
| a. Hay 2 casos: | b. La lista xs puede ser: | c. El árbol binario t puede ser: |
| <ul style="list-style-type: none"> altura i ≥ altura d altura i < altura d | <ul style="list-style-type: none"> [] [x] x:y:ys | <ul style="list-style-type: none"> Nil Bin i r d |
| (Con i, d :: AB a). | (Con x, y :: a, ys :: [a]). | (Con r :: a, i y d :: AB a). |

Respuesta: C

Pregunta 5: Definir el predicado unario P que se podría usar para demostrar la siguiente propiedad por inducción estructural:

$\forall xs :: [a]. \forall ys :: [a]. \text{length} (\text{entrelazar } xs \text{ } ys) = \text{length } xs + \text{length } ys$

Respuesta: $P(xs) = \forall ys :: [a]. \text{length} (\text{entrelazar } xs \text{ } ys) = \text{length } xs + \text{length } ys$
Para demostrar la propiedad, se demuestra que vale $P(xs)$.
 $\forall xs :: [a]$

Pregunta 6: Si se quisiera demostrar que vale $\neg P \vee Q \vdash Q \vee (P \Rightarrow Q)$ por deducción natural, ¿cuál sería el error al aplicar el siguiente paso?

$$\frac{\neg P \vee Q \vdash Q}{\neg P \vee Q \vdash Q \vee (P \Rightarrow Q)} (V_{i1})$$

en el contexto.

Respuesta: El error es que pierdo información necesaria para poder demostrarlo, ya que con $\neg P \vee Q$ no tengo forma de probar que siempre vale Q .

Pregunta 7: Indicar a qué término reduce en un paso la siguiente expresión, y mediante qué regla(s):
if isZero(zero) then Pred(Succ(zero)) else zero

Respuesta: reduce a if true then Pred(Succ(zero)) else zero, por reglas E-IF y E-IsZero.

Pregunta 8: Sea la extensión del Cálculo Lambda con listas vista en la guía:

$$M ::= \dots \mid []_\tau \mid M :: M \mid \text{case } M \text{ of } \{ [] \rightsquigarrow M \mid h :: t \rightsquigarrow M \} \mid \text{foldr } M \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow M$$

Indicar el error en la siguiente regla de cómputo para foldr:

$$\frac{\text{foldr } V_1 :: V_2 \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow N \rightarrow N}{\text{foldr } V_1 :: V_2 \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow N} \text{ (E-FOLDR-CONS)}$$

Respuesta: El término N puede tener apariciones libres de las variables h y r , por lo que al reducir el foldr $V_1 :: V_2$ es necesario ligarlas con la cabeza y el resultado de la recursión. La regla debería ser:

$$\text{foldr } V_1 :: V_2 \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow N \longrightarrow * \\ * N \{ h := V_1, r := \text{foldr } V_2 \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow N \}$$